DL_PY2F — Reference Manual (v1.4.1)

20 July 2026

Contents

Overview
- Licence
- Getting DL_PY2F
  - Method 1: from source (recommended)
  - Method 2: PyPI (recommended)
  - Method 3: Debian PPA
  - Launching
- Memory ownership (core principle)

Part 1 — Python-to-Fortran interoperability
- 1.1 Getting started
  - Accessing Python data from existing Fortran code
- 1.2 Workflow
  - Mode A: lazy mode (alias, no copies)
  - Mode B: safe mode (explicit copy and set)
- 1.3 Callback functions
  - Python side
  - Accessing the owning object
  - Fortran side
  - Interaction with lazy mode
  - Lifetime and error handling
- 1.4 Advanced class features
  - Class attributes reference
  - The @objutils.init() decorator
  - Structured arrays in _kwargs and _fields
  - Nested objects
  - Scalars and mutability
  - The size setter and _mega resizing
- 1.5 API reference
  - py2f — the Python-to-Fortran conversion function
  - ptr2dict — handle-to-dictionary conversion
  - dictType%finalise — release metadata
  - dictType%get — generic getter (16 specific procedures)
  - dictType%set — generic setter (8 specific procedures)
  - Introspection routines
- 1.6 Data model
  - Python types
  - NumPy arrays
- 1.7 Memory ownership

Part 2 — Fortran-to-Python interoperability
- 2.1 Getting started

2.2 Data model
Supported intrinsic types
Supported array ranks
Supported Fortran attributes
Linked lists
Arrays, contiguity, and strides
Memory ownership
2.3 Initialisation and teardown
2.4 API reference
DL_DL — shared-library loader
Byte-level setters (static methods on DL_DL)
DL_DT — derived-type proxy
Part 3 — Shared reference
3.1 Error handling and diagnostics
3.2 Compatibility and limitations
3.3 MPI / multi-instance usage
Model A — one Python interpreter per MPI rank
Model B — single Python interpreter with MPI distribution
General notes
3.4 Common pitfalls and reminders
3.5 Utility modules (dl_py2f.utils)
objutils — object initialisation
nputils — NumPy helpers
modutils — shared-library loading
fileutils — file operations
miscutils — miscellaneous helpers
strutils — string helpers
dictutils — dictionary helpers
iterutils — iteration and flattening helpers
ctypeutils — ctypes byte conversion
colours — ANSI terminal colour constants

Overview

`DL_PY2F` is an open-source library for creating modern Python interfaces and NumPy-based data structures around existing Fortran applications, giving both sides direct access to the same data without copies where possible. It has been used in production to drive ChemShell. This document is the reference for application developers using `DL_PY2F`, covering how to set up and use the library, supported data types and array ranks, memory ownership, MPI patterns, and known limitations.

DL_PY2F provides two interoperability directions:

Python-to-Fortran (primary, production ready). You define your data in a Python class, and Fortran accesses it by name through a dictionary-like interface (`dictType`). Zero-copy is used wherever possible. The Python side loads the compiled Fortran `.so` via `ctypes.CDLL` ; the Fortran side uses DL_PY2F's source-level API (`use DL_PY2F` , `dictType` , `get` / `set`). The library never edits or regenerates Fortran sources, nor does it generate Python wrapper code (unlike tools such as F2PY).

Fortran-to-Python (beta, gfortran, ifx, Flang, and nvfortran). Python loads an existing Fortran `.so` and parses `.mod` metadata to expose module variables, procedures, and derived types with dot syntax, without any changes to the Fortran source.

The reference is organised into three parts: Part 1 covers Python-to-Fortran interoperability, Part 2 covers Fortran-to-Python interoperability, and Part 3 collects material shared by both directions (MPI, errors, compatibility, pitfalls). Application developers working in only one direction can read the relevant part without wading through the other.

Licence

DL_PY2F is open source and released under the GNU Lesser General Public License v3.0 (LGPLv3).

Getting DL_PY2F

DL_PY2F can be obtained in three ways. In all cases the result is a shared library `libdl_py2f.so` and a Fortran module file `dl_py2f.mod` that your application links against at buildtime, plus a Python package `dl_py2f` that you import at runtime.

Both the Python-to-Fortran and Fortran-to-Python paths work with all listed compilers (gfortran, ifx, Flang, nvfortran). See section 3.2 for the full compiler compatibility matrix and minimum version requirements.

Method 1: from source (recommended)

Clone the DL_PY2F repository into your application's source tree and build it together with your code using CMake. This is the most flexible approach because it compiles DL_PY2F with the same compiler and compiler options as your application. Since DL_PY2F is

licensed under LGPLv3, application developers are free to redistribute it alongside their own projects.

Project structure. A recommended layout for a DL_PY2F-based project is:

```
my_project/
├── my_app/
│   ├── __init__.py       # package init: imports, doSomething()
│   ├── my_app.py         # Python class (ctypes.Structure + _kwargs)
│   ├── callback.py       # callback function(s) invoked from Fortran
│   ├── my_app_dirs.py.in # CMake-configured paths (lib and module dirs)
│   ├── CMakeLists.txt    # build file
│   ├── interface.f90     # Fortran bind(c) interface using DL_PY2F
│   ├── my_code/          # my existing Fortran codebase
│   │   ├── my_code.f90
│   │   └── my_module.f90
│   └── dl_py2f/          # clone DL_PY2F here
└── user_script.py        # end-user entry point
```

The `__init__.py` is the entry point to your package. It loads the compiled shared library, calls `py2f()`, and invokes the Fortran interface:

```
import ctypes, os.path
import dl_py2f
from .my_app      import MyApp
from .my_app_dirs import libdir

def doSomething(obj):
    libapp = ctypes.CDLL(os.path.join(libdir, 'libmy_app.so'))
    handle = dl_py2f.py2f(obj, byref=True)
    ierror = libapp.interface_my_app(handle)
    return ierror
```

The end-user script (`user_script.py`) imports your package, creates an instance, and calls the function:

```
from my_app import MyApp, doSomething

app = MyApp()
app.size = 10
ierror = doSomething(app)
```

A separate user script is not strictly required. For simpler projects you can define a `__main__` block in `__init__.py` as the entry point instead.

Compiling. Set the compilers via environment variables before running CMake:

```
# GNU
$ export FC=gfortran; export CXX=g++
# OR Intel
$ export FC=ifx; export CXX=icpx
# OR Flang/Clang++
$ export FC=flang; export CXX=clang++
# OR NVIDIA HPC
$ export FC=nvfortran; export CXX=nvc++
```

Clone DL_PY2F into the source tree and compile:

```
$ cd my_project/my_app
$ git clone https://github.com/stfc/dl_py2f.git dl_py2f

$ cmake -S . -B build -DFROM_SOURCE:BOOL=TRUE
$ cmake --build build
```

There are two recommended approaches for organising the CMake build.

Option A (recommended for large Fortran codebases): compile your existing Fortran code as one or more static libraries (`.a` archives) with `-fPIC` (required because they will be linked into a shared library), then compile only the thin interface layer (`interface.f90`) as a shared library (`.so`) and link the static archives into it. This keeps the build modular and avoids recompiling your entire codebase when only the interface changes:

```
cmake_minimum_required(VERSION 3.16)
project(My_Code LANGUAGES Fortran)

set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${PROJECT_BINARY_DIR}/lib)
set(CMAKE_Fortran_MODULE_DIRECTORY ${PROJECT_BINARY_DIR}/modules)

# my thin interface layer
add_library(my_app SHARED interface.f90)

# link my existing Fortran code as a static archive
target_link_options(my_app PRIVATE "LINKER:--whole-archive")
target_link_options(my_app PRIVATE ${YOUR_CODE_DIR}/libmy_code.a)
target_link_options(my_app PRIVATE "LINKER:--no-whole-archive")

# compile DL_PY2F from source
if(FROM_SOURCE)
    add_subdirectory(${CMAKE_CURRENT_LIST_DIR}/dl_py2f)
    include_directories(${CMAKE_CURRENT_LIST_DIR}/dl_py2f)
endif()

target_link_libraries(my_app PRIVATE dl_py2f)
```

Option B (simpler, suitable for smaller projects): compile all Fortran source files into a single shared library:

```
cmake_minimum_required(VERSION 3.16)
project(My_Code LANGUAGES Fortran)

set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${PROJECT_BINARY_DIR}/lib)
set(CMAKE_Fortran_MODULE_DIRECTORY ${PROJECT_BINARY_DIR}/modules)

add_library(my_app SHARED interface.f90
                          my_code/my_code.f90
                          my_code/my_module.f90)

if(FROM_SOURCE)
    add_subdirectory(${CMAKE_CURRENT_LIST_DIR}/dl_py2f)
    include_directories(${CMAKE_CURRENT_LIST_DIR}/dl_py2f)
endif()

target_link_libraries(my_app PRIVATE dl_py2f)
```

Option C (when the existing Fortran code is only available as a shared library): if the existing codebase provides a `.so` rather than static archives, circular dependencies can

arise when the interface layer and the existing library depend on each other's symbols (e.g., the interface calls into the existing `.so`, which in turn calls back into the interface's callback routines). One workaround is to compile the interface layer as a static library, embed it into a thin wrapper `.so` via `--whole-archive`, and link that wrapper against the existing `.so`. However, symbol overriding between the two shared libraries is not guaranteed by the linker. The preferred solutions are either to obtain static archives from the existing codebase, or to build everything as a single project.

For complex projects, it is recommended to use a linker version script (e.g., `export.map`) to control which symbols are exported from the `.so` and prevent name clashes between libraries.

Method 2: PyPI (recommended)

Install into a Python virtual environment. This is the simplest approach, requiring only internet access and `pip`:

```
$ python3 -m venv .venv && source .venv/bin/activate
$ pip install dl_py2f
```

With DL_PY2F installed via pip, CMake finds the library and module file under the virtual environment's `site-packages/dl_py2f/` directory. No `FROM_SOURCE` flag is needed.

Method 3: Debian PPA

gfortran only, because Debian PPA builds with free toolchains and the precompiled `.mod` file shipped is in gfortran format:

```
$ sudo add-apt-repository ppa:dl-py2f/ppa
$ sudo apt update && sudo apt install dl-py2f
```

Note: the package name on Launchpad is `dl-py2f` (with a hyphen) because Debian PPA does not allow underscores. With DL_PY2F installed system-wide, your `CMakeLists.txt` does not need `FROM_SOURCE` or `add_subdirectory`; CMake will find the library and module file in the standard system paths.

Launching

After building (regardless of which installation method was used), run the Python script with `PYTHONPATH` pointing to the project root. If DL_PY2F was installed via pip (Method 2), activate the virtual environment first:

```
$ cd my_project
$ source .venv/bin/activate   # if using a virtual environment
$ export PYTHONPATH=$PWD
$ python3 user_script.py
```

Memory ownership (core principle)

Allocation and deallocation remain the responsibility of the side that created the memory. DL_PY2F never frees memory it did not allocate. A handle is valid only while the underlying allocation exists; beyond that, behaviour is undefined. Direction-specific ownership rules are documented in section 1.7 and section 2.2.

Part 1 — Python-to-Fortran interoperability

The application developer defines data in Python and passes it to Fortran through a minimal `bind(c)` interface, requiring minimal or no modifications to the original Fortran source. Callback functions (section 1.3) further enable the developer to inject Python routines, such as machine-learning models or custom analysis routines, into the Fortran execution flow at runtime.

1.1 Getting started

Python side: define a class inheriting from `ctypes.Structure` with a `_kwargs` dictionary. Each key in `_kwargs` becomes an entry that Fortran can retrieve by name. Scalars (`int` , `float` , `bool` , `str`) are passed by value; NumPy arrays are passed by reference (zero-copy when contiguous). Keys starting with `_` are skipped.

```
import ctypes
import numpy
import dl_py2f

class MyApp(ctypes.Structure):
    _kwargs = {
                'natoms' :  0,
                'energy' :  numpy.zeros( 1,    dtype=numpy.float64),
                'coords' :  numpy.zeros((1,3), dtype=numpy.float64),
              }

app = MyApp()
app.natoms = 3
app.coords = numpy.array([[1.0, 2.0, 3.0],
                          [4.0, 5.0, 6.0],
                          [7.0, 8.0, 9.0]], dtype=numpy.float64)
```

Load the compiled Fortran shared library, convert the object with `py2f()` , and call the Fortran entry point. Ensure the Python object and its NumPy arrays remain alive for the entire Fortran call because DL_PY2F does not extend their lifetime.

```
libapp = ctypes.CDLL('/path/to/libexample.so')
handle = dl_py2f.py2f(app)
ierror = libapp.interface_example(handle)

# results are now available in app.energy, app.coords, etc.
print(app.energy)
```

Fortran side: receive the handle as `type(PyType), intent(in)` , convert it to a `dictType` with `ptr2dict` , and use `get` / `set` to access entries by name:

```
integer(c_int) function interface_example(objPtr) bind(c)

    use iso_c_binding
    use DL_PY2F, only: PyType, dictType, ptr2dict

    type(PyType)            , intent(in) :: objPtr

    type(dictType) , pointer             :: PyObj
    integer(c_long)                      :: natoms
    real(kind=8)   , pointer             :: energy(:), coords(:,:)

    ! convert handle to a dictionary
    allocate(PyObj, source=ptr2dict(objPtr))

    ! retrieve data by name
    call PyObj%get('natoms', natoms)
    call PyObj%get('energy', energy)        ! zero-copy alias
    call PyObj%get('coords', coords)        ! zero-copy alias

    ! compute and write results in place
    call compute_energy(coords, energy)
    coords    = coords * 2.0

    ! clean up: release metadata, nullify pointers (never deallocate aliased arrays)
    call PyObj%finalise()
    deallocate(PyObj)
    nullify(energy, coords)
    interface_example = 0

endfunction interface_example
```

After the Fortran function returns, the Python side sees the updated values in `app.energy` and `app.coords` because the arrays were aliased (zero-copy). On the Fortran side, clean up with `finalise` + `deallocate` on the dictionary and `nullify` on the pointers. On the Python side, normal garbage collection applies.

Accessing Python data from existing Fortran code

In practice, it is common to place the interface function inside a Fortran module and make `PyObj` a public module-level variable:

```
module ExampleModule

    use DL_PY2F, only: dictType

    type(dictType), pointer, public :: PyObj

    contains

    integer(c_int) function interface_example(objPtr) bind(c)
        ! function body
    endfunction interface_example

endmodule ExampleModule
```

This allows helper routines in separate files (compiled into the same `.so`) to get and set Python data at any point during the Fortran execution, not only at the entry point or through callbacks:

```
subroutine get_coords(coords)

    use ExampleModule, only: PyObj

    real(kind=8), pointer, intent(out) :: coords(:,:)

    call PyObj%get('coords', coords)

endsubroutine get_coords

subroutine set_coords(coords)

    use ExampleModule, only: PyObj

    real(kind=8),          intent(in)  :: coords(:,:)

    call PyObj%set('coords', coords)

endsubroutine set_coords
```

The existing Fortran codebase can then call these routines directly, requiring minimal or no modifications:

```
call get_coords(coords)
call do_computation(coords, energy)
call set_coords(coords)
```

This keeps DL_PY2F-specific code (`%get`, `%set`) confined to the helper routines. The original codebase has no dependency on DL_PY2F, can be compiled and tested independently, and remains unchanged if the interoperability layer is ever replaced.

1.2 Workflow

The Python side is the same regardless of which access mode is used on the Fortran side:

```
class App(ctypes.Structure):
    _kwargs = {'energy' : numpy.zeros( 1,     dtype=numpy.float64),
               'forces' : numpy.zeros((10,3), dtype=numpy.float64)}

app = App()

handle = dl_py2f.py2f(app, byref=True)
ierror = libapp.interface_app(handle)
```

The array sizes here are fixed. For resizable arrays backed by `_mega`, see section 1.4.

On the Fortran side, you choose per array how to access the data:

- **Lazy (alias) mode** — `PyObj%get('coords', twodimdbl)` yields a pointer into the NumPy buffer. Mutate it in place; never `deallocate` it, `nullify` the pointer instead.
- **Safe (copy then set) mode** — `get(..., readonly=.true.)` copies into a Fortran-allocated buffer. After use, push changes back with `PyObj%set('coords', buffer)` and then `deallocate` the buffer.

On Fortran exit, call `PyObj%finalise()`, then `deallocate(PyObj)`, and `nullify` any remaining pointers. Never free Python-owned arrays from Fortran.

Mode A: lazy mode (alias, no copies)

In this mode the Fortran pointers alias the Python buffers directly. Never `deallocate` them.

```
integer(c_int) function interface_app(objPtr) bind(c)

    use iso_c_binding
    use DL_PY2F, only: PyType, dictType, ptr2dict

    type(PyType), intent(in) :: objPtr
    type(dictType), pointer  :: PyObj
    real(kind=8), pointer    :: energy(:), forces(:,:)

    allocate(PyObj, source=ptr2dict(objPtr))

    ! alias into NumPy arrays (zero-copy)
    call PyObj%get('energy', energy)
    call PyObj%get('forces', forces)

    ! modify in place; changes are reflected on the Python side immediately
    call compute_energy(forces, energy)
    forces    = forces * 2.0

    ! teardown: finalise metadata, never deallocate aliased data
    call PyObj%finalise()
    deallocate(PyObj)
    nullify(energy, forces)

endfunction interface_app
```

`energy` and `forces` point directly into the NumPy buffers. No copies are made, and modifications are reflected on the Python side immediately. If Fortran `deallocate`s an aliased pointer, it corrupts Python memory; always use `nullify` instead.

Mode B: safe mode (explicit copy and set)

In this mode the developer allocates separate Fortran buffers and explicitly controls when data is read from and written back to Python.

```
integer(c_int) function interface_app(objPtr) bind(c)

    use iso_c_binding
    use DL_PY2F, only: PyType, dictType, ptr2dict

    type(PyType), intent(in)  :: objPtr
    type(dictType), pointer   :: PyObj
    real(kind=8), allocatable :: energy(:), forces(:,:)
    integer                   :: shp(2)

    allocate(PyObj, source=ptr2dict(objPtr))

    ! query shape and allocate Fortran-owned buffers
    shp = PyObj%enquireShape('forces')
    allocate(forces(shp(1),shp(2)), energy(1))

    ! copy from Python into local buffers (isolated from Python)
    call PyObj%get('forces', forces, readonly=.true.)
    call PyObj%get('energy', energy, readonly=.true.)

    ! modify the local copy
    call compute_energy(forces, energy)
    forces    = forces * 2.0

    ! push modified copy back to Python
    call PyObj%set('forces', forces)
    call PyObj%set('energy', energy)

    ! teardown: finalise metadata, deallocate Fortran buffers
    call PyObj%finalise()
    deallocate(PyObj)
    deallocate(forces, energy)

endfunction interface_app
```

`readonly=.true.` forces a copy from Python. Changes reach Python only if you call `%set` afterwards. This mode is safer against accidental corruption, at the cost of the extra copy.

1.3 Callback functions

Callback functions allow Fortran to invoke Python code at runtime. This makes it possible to inject Python functionality, for example a machine-learning model or a custom analysis routine, into the Fortran execution flow with minimal changes to the existing codebase. On the Python side, a callable is placed in `_kwargs` and converted to a C function pointer by `py2f()`. On the Fortran side, the pointer is retrieved with `dictType%get` and bound to a procedure pointer using `c_f_procpointer`.

Python side

Place the callable in `_kwargs`:

```
_kwargs = {
            'callback' : callback,
            ...
          }
```

`py2f()` wraps the callable as a C function pointer via `ctypes.CFUNCTYPE`. The return type defaults to `c_long` (`INTEGER(8)`); if the callable has type hints, `py2f()` inspects them to select the appropriate return type (e.g., `float` maps to `c_double`).

Accessing the owning object

To allow the callback to access the owning object's data when invoked from Fortran, attach the object to the function as an attribute:

```
callback.obj = self   # 'obj' is an arbitrary attribute name
```

Inside the callback body, `callback.obj` provides access to all attributes of the owning instance:

```
def callback(*args) -> int:
    callback.obj.energy[:] *= 1.1
    callback.obj.forces[:] *= 1.0
    callback.obj.doSomething()
    return 0
```

Fortran side

Declare an abstract interface matching the callback's C signature, then retrieve and bind:

```
! declare the expected C signature
abstract interface
    integer(c_long) function callback() bind(c)
        use iso_c_binding
    endfunction callback
endinterface
procedure(callback), pointer :: PyCallback
type(c_funptr)               :: pyfuncPtr

! retrieve and bind the Python callback
call PyObj%get('callback', pyfuncPtr)
call c_f_procpointer(pyfuncPtr, PyCallback)

! invoke it
ierror = PyCallback()
```

The `getCFuncPtr` specific procedure of `dictType%get` resolves the `c_funptr` target. See section 1.5 for the full generic getter table.

Interaction with lazy mode

When arrays are retrieved in lazy (alias) mode before the callback is invoked, mutations made by the Python callback are immediately visible in the Fortran pointers:

```
! alias into the Python buffers (zero-copy)
call PyObj%get('energy', energy)
call PyObj%get('forces', forces)

! Python mutates energy/forces in place
ierror = PyCallback()

! energy and forces now reflect the Python-side changes
call compute_energy(forces, energy)
```

In safe mode (`readonly=.true.`), the Fortran copy is isolated from the callback's changes. To pick up values modified by the callback, call `%get` again after the callback returns. See section 1.2 Mode B for an example.

Lifetime and error handling

- Keep the Python callable and its owning object alive for as long as the Fortran side may invoke the callback. `py2f()` does not extend lifetimes.
- If a Python exception is raised inside the callback, catch it within the callback itself. An uncaught exception cannot propagate back through Fortran and will crash the program.

1.4 Advanced class features

Section 1.1 shows only `_kwargs`. For larger applications, especially those with resizable arrays, structured data, or class hierarchies, DL_PY2F provides additional class attributes that automate initialisation and manage a columnar backing store.

Class attributes reference

`_kwargs` (required, `dict`). A dictionary mapping attribute names to their default values and data types. The default value determines the type that `py2f()` uses when converting the entry for Fortran, so the actual value assigned at runtime should match the default's type. Every entry becomes a field visible to Fortran after `py2f()` processes the object. Keys beginning with `_` are skipped. See section 1.6 for the full list of supported types.

`_fields` (optional, sequence, namely `tuple` or `list`, of `str`). A tuple or list of attribute names that are backed by a single contiguous `RecArray` (`_mega`). Every field listed here must use 8-byte-aligned dtypes (`float64`, `int64`, `S8`, or a structured dtype whose sub-fields are themselves 8-byte-aligned) so the record layout stays stable across C and Fortran. Structured arrays (see below) can be included in `_fields`. Arrays in `_kwargs` that are not listed in `_fields` remain independent, they are still passed to Fortran but are not part of `_mega` and can have any shape.

`_mega` (auto-created, `RecArray`). A `RecArray` (a `numpy.recarray` subclass from `dl_py2f.utils.nputils`) that holds the columnar data for all entries in `_fields`. It is a container array where all fields share the same first dimension, which makes it convenient for particle-based simulations and similar applications where many arrays (coordinates, charges, tags, etc.) must be resized together. Resizing `_mega` with `expand()` resizes all fields at once (e.g., `self._mega.expand(100)`). It is created automatically by the `@dl_py2f.utils.objutils.init()` decorator (see below). Do not create `_mega` manually.

`_priorities` (optional, sequence, namely `tuple` or `list`, of `str`). Controls the order in which `_kwargs` attributes are initialised when they depend on one another. Attributes listed earlier are initialised first. For example, if setting `coords` depends on `natoms` having been set, list `natoms` before `coords`.

`_init` (optional, sequence, namely `tuple` or `list`, of `str`). Used by the `@objutils.init()` decorator to determine which class attributes to process during initialisation (not used by `py2f()`). Each entry can be either a `dict` (e.g., `_kwargs`, `_internals`), whose key-value pairs are initialised as instance attributes, or a sequence (e.g., `_fields`), which is flattened and stored as an instance attribute. Multiple types can be mixed in the same `_init` tuple. For example:

```
_init = '_kwargs', '_internals', '_fields'
```

This initialises all entries from `_kwargs` and `_internals` as instance attributes, and flattens `_fields` into a single sequence on the instance. The ordering within each dict is controlled by `_priorities`.

`_inherit` (optional, sequence, namely `tuple` or `list`, of `str`). Names class attributes that should be merged from parent classes. The `@objutils.init()` decorator concatenates sequences (tuples or lists) and merges dictionaries from the parent chain automatically. This is how a subclass can extend `_fields` or `_priorities` without repeating the parent's entries. For example:

```
class Base(ctypes.Structure):
    _fields  = 'coords', 'charges'
    _inherit = '_fields',

class Child(Base):
    _fields = 'velocities',    # only the new field
# after @objutils.init(), Child instance has
# _fields = ('coords', 'charges', 'velocities')
```

`synons` (optional, `dict`). `py2f()` checks `obj.synons` and, for each matching key, uses the mapped name on the Fortran side instead of the original key. Multiple Python names can map to the same Fortran name, which lets the application developer create convenient synonyms for end users of the application. For example:

```
synons = {'natoms': 'npts', 'number_of_atoms': 'npts', 'coords': 'xyz'}
```

Here both `natoms` and `number_of_atoms` on the Python side map to `npts` on the Fortran side, so end users can use whichever name is more natural in their context. `coords` maps to `xyz`. If `synons` is not defined, `py2f()` uses the original `_kwargs` key names. Note: `synons` will be renamed to `_synons` in a future version for consistency with other class attributes.

`_internals` (optional, `dict`). Attributes that live only on the Python side. When `_internals` is listed in `_init`, the `@objutils.init()` decorator initialises its entries as member attributes of the instance, just like `_kwargs` entries. However, because `py2f()` reads only `_kwargs`, anything in `_internals` is invisible to Fortran.

The `@objutils.init()` decorator

Apply `@objutils.init()` (from `dl_py2f.utils.objutils`) to the class `__init__` method. It runs three steps before the decorated body executes:

1. **Inherit**: merges sequence and dict attributes from parent classes listed in `_inherit`.
2. **Initialise**: processes each dict listed in `_init`, respecting `_priorities` ordering. If a key has a property setter, the decorator delegates to it.
3. **Run**: executes the original `__init__` body.

Structured arrays in `_kwargs` and `_fields`

A structured array (NumPy `recarray` or void-dtype array) can be placed in `_kwargs` and optionally in `_fields`. When `py2f()` encounters one, it unpacks each named sub-field into a separate Fortran array, each retrievable by name via `dictType%get`. For example, a Z-matrix with sub-fields `j`, `bond`, `k`, `angle`, `l`, `dihedral`:

```
from numpy import zeros, dtype, int64, float64

_kwargs = {
            'zmatrix' : zeros(shape=(1,), dtype=dtype([('j',        int64),
                                                       ('bond',     float64),
                                                       ('k',        int64),
                                                       ('angle',    float64),
                                                       ('l',        int64),
                                                       ('dihedral', float64)])),
          }
_fields = 'coords', 'tags', 'zmatrix'   # zmatrix is part of _mega
```

On the Python side, populate sub-fields with dictionary-style or attribute access:

```
app.zmatrix['bond']     = 1.5    # or app.zmatrix.bond = 1.5
app.zmatrix['angle']    = 109.5  # or app.zmatrix.angle = 109.5
app.zmatrix['dihedral'] = 180.0  # or app.zmatrix.dihedral = 180.0
```

On the Fortran side, each sub-field is a separate 1D array:

```
real(kind=8), pointer :: bond(:), angle(:), dihedral(:)
call PyObj%get('bond',     bond)
call PyObj%get('angle',    angle)
call PyObj%get('dihedral', dihedral)
```

Nested objects

When a `_kwargs` entry is an object that itself has a `_kwargs` attribute, `py2f()` recursively converts it. This allows unlimited nesting depth. On the Fortran side, retrieve the nested handle with `dictType%get` into a `type(PyType)` pointer, then convert it to its own `dictType` with `ptr2dict`.

Python side:

```
class GrandChild(ctypes.Structure):
    _kwargs = { 'natoms' : 0 }

class Child(ctypes.Structure):
    _kwargs = { 'grandchild' : GrandChild() }

class Parent(ctypes.Structure):
    _kwargs = { 'child' : Child() }

parent = Parent()
parent.child.grandchild.natoms = 42
handle = dl_py2f.py2f(parent)
```

Fortran side:

```
type(dictType), pointer :: PyObj, ChildObj, GrandChildObj
type(PyType),   pointer :: childPtr, grandchildPtr

allocate(PyObj, source=ptr2dict(objPtr))

! retrieve the nested child handle
call PyObj%get('child', childPtr)
allocate(ChildObj, source=ptr2dict(childPtr))

! go one level deeper
call ChildObj%get('grandchild', grandchildPtr)
allocate(GrandChildObj, source=ptr2dict(grandchildPtr))

call GrandChildObj%get('natoms', natoms)   ! natoms = 42

! clean up each level
call GrandChildObj%finalise(); deallocate(GrandChildObj)
call ChildObj%finalise();      deallocate(ChildObj)
call PyObj%finalise();         deallocate(PyObj)
```

Any object without a `_kwargs` attribute is silently skipped by `py2f()`.

Scalars and mutability

Python scalars (`int`, `float`, `bool`) in `_kwargs` are immutable objects. Once `py2f()` takes a snapshot, changing the Python attribute does not update the Fortran side, and Fortran cannot write back to a scalar either. There is currently no mechanism to make a true scalar mutable across the Python-Fortran boundary.

The practical workaround is to use a **one-element array** instead of a scalar for any value that Fortran needs to modify:

```
_kwargs = {
            'energy': numpy.zeros(1, dtype=numpy.float64), # mutable: one-element array
            'scale' : 1.23456,                             # immutable: true scalar
          }
```

On the Fortran side, `energy` is a 1D array of length 1 and can be read and written like any other array. `scale` is a read-only snapshot.

The `size` setter and `_mega` resizing

A common pattern is a `size` (or `natoms`) property that controls `_mega.shape[0]`, the number of rows shared by all fields in `_mega`, so that all fields (coordinates, tags, charges, etc.) are resized together:

```
@property
def size(self):
    return self._mega.shape[0]

@size.setter
def size(self, val):
    try:
        self._mega.expand(val)
    except:
        dt = dl_py2f.utils.nputils.getDTypeFromObj(self)  # see section 3.5
        self._mega = dl_py2f.utils.nputils.RecArray(shape=(val,), dtype=dt)
```

The following example brings together the features described above. Note that `c` is an independent array not listed in `_fields`, it is passed to Fortran but is not part of `_mega` and can have any shape:

```
import ctypes
import numpy
from dl_py2f               import py2f
from dl_py2f.utils         import objutils
from dl_py2f.utils.nputils import RecArray

def callback(*args) -> int:
    callback.obj.energy[:] *= 1.1
    callback.obj.forces[:] *= 1.0
    return 0

class MyApp(ctypes.Structure):
    _kwargs = {
                'natoms'  : 0,
                'energy'  : numpy.zeros( 1,    dtype=numpy.float64),
                'forces'  : numpy.zeros((1,3), dtype=numpy.float64),
                'coords'  : numpy.zeros((1,3), dtype=numpy.float64),
                'tags'    : numpy.zeros( 1,    dtype=numpy.int64),
                'callback': callback,
                'c'       : numpy.zeros( 5,    dtype=numpy.int64),   # independent from
_mega
                'zmatrix' : numpy.zeros( 1,    dtype=numpy.dtype([
                            ('j', numpy.int64), ('bond', numpy.float64),
                            ('k', numpy.int64), ('angle', numpy.float64),
                            ('l', numpy.int64), ('dihedral', numpy.float64)])),
              }

    _fields     = 'energy', 'forces', 'coords', 'tags', 'zmatrix'   # 'c' is NOT here
    _priorities = 'coords', 'tags'
    _init       = ('_kwargs',)
    _inherit    = ()
    _internals  = {}
    synons      = {}

    @objutils.init()
    def __init__(self):
        callback.obj = self

app = MyApp()
app.natoms = 10
app.coords = numpy.random.rand(10, 3)
app.zmatrix['bond'] = 1.5  # or app.zmatrix.bond = 1.5

handle = py2f(app, byref=True)
```

1.5 API reference

`py2f` — the Python-to-Fortran conversion function

- **`py2f(obj, debug=0, byref=False)`** Converts the object's `_kwargs` dictionary into a handle that Fortran can consume via `ptr2dict`. Each entry is automatically converted to the appropriate `ctypes` representation (see the type tables in section 1.6). The debug levels are:
- `0` — silent.
- `> 2` — prints the object type and attribute summary.
- `> 4` — prints per-field type and pointer details.

Both `byref=False` (default) and `byref=True` produce a handle that can be passed to a Fortran `bind(c)` function. The difference is in what `py2f()` returns:

- `byref=False` (default) — returns the `ctypes.Structure` instance. You can store it, inspect it, and pass it to Fortran directly (ctypes passes structures by reference automatically): `python handle = dl_py2f.py2f(app) ierror = libapp.interface_example(handle)`
- `byref=True` — returns a `ctypes.byref()` pointer. This is a temporary object that cannot be stored or reused, so it is only suitable for immediate one-shot calls: `python ierror = libapp.interface_example(dl_py2f.py2f(app, byref=True))`

`ptr2dict` — handle-to-dictionary conversion

- **`ptr2dict(metaPtr) -> dictType`** (Fortran) Converts the handle received from Python into a `dictType`. This is the required first step on the Fortran side before any `%get` or `%set` calls. The caller must later `finalise` and `deallocate`.

`dictType%finalise` — release metadata

- **`dictType%finalise()`** Releases all Fortran-side metadata (the linked-list nodes) associated with the `dictType`. It never deallocates the underlying array data — ownership of that stays with Python (or with you, if you allocated your own buffers). Alias-mode pointers obtained via `%get` with `readonly=.false.` remain valid after `finalise` because they point directly into Python memory, not into the `dictType` metadata. You should still `nullify` them once you are finished, but they are not immediately dangling.

`dictType%get` — generic getter (16 specific procedures)

The `get` generic resolves to one of the following based on the target argument's type and rank:

Specific procedure	Target type	Notes
`getInt`	`integer(4)`	Scalar, copy by value
`getIntCLong`	`integer(8)` / `integer(c_long)`	Scalar, copy by value
`getReal`	`real(4)` / `real(c_float)`	Scalar, copy by value
`getDouble`	`real(8)` / `real(c_double)`	Scalar, copy by value
`getOneDimDbl`	`real(8), pointer :: (:)`	1D double array; `readonly` arg
`getTwoDimDbl`	`real(8), pointer :: (:,:)`	2D double array; `readonly` arg
`getOneDimLng`	`integer(8), pointer :: (:)`	1D int64 array; `readonly` arg
`getTwoDimLng`	`integer(8), pointer :: (:,:)`	2D int64 array; `readonly` arg
`getOneDimInt`	`integer(4), pointer :: (:)`	1D int32 array; `readonly` arg
`getTwoDimInt`	`integer(4), pointer :: (:,:)`	2D int32 array; `readonly` arg
`getChar`	`character(len=:), pointer`	Character string, pointer to internal data
`getCCharArray`	`character(len=1) :: (:)`	C-style char array, copy by value
`getLogical`	`logical`	Fortran logical, copy by value
`getCLogical`	`logical(c_bool)`	C-interoperable logical
`getPyPtr`	`type(PyType), pointer`	Nested object handle
`getCFuncPtr`	`type(c_funptr)`	Callback function pointer

For array targets, the optional `readonly` argument (default `.false.`) controls aliasing: `.false.` yields a pointer into the Python buffer (zero-copy); `.true.` copies the data into a Fortran-allocated buffer.

`dictType%set` — generic setter (8 specific procedures)

Specific procedure	Value type	Notes
`setScalar`	`class(*)`	Does not propagate to the Python side; effectively unusable. Use a one-element array instead
`setOneDimDbl`	`real(8) :: (:)`	1D double array, copies to Python
`setTwoDimDbl`	`real(8) :: (:,:)`	2D double array, copies to Python
`setOneDimLng`	`integer(8) :: (:)`	1D int64 array, copies to Python
`setTwoDimLng`	`integer(8) :: (:,:)`	2D int64 array, copies to Python
`setOneDimInt`	`integer(4) :: (:)`	1D int32 array, copies to Python
`setTwoDimInt`	`integer(4) :: (:,:)`	2D int32 array, copies to Python
`setCCharArray`	`character(len=1) :: (:)`	C-style char array

Because Python scalars (`float` , `int`) are immutable, updating a scalar value from Fortran requires a one-element array combined with `%set` :

```
! Python float is immutable; use a one-element array + set
call PyObj%get('energy', onedimdbl, readonly=.true.)
onedimdbl = onedimdbl * 1.5
call PyObj%set('energy', onedimdbl)
```

Introspection routines

- **`dictType%keys()`** — prints all keys to standard output. Useful for debugging. Will be renamed `%printKeys` in a future version.
- **`dictType%enquireKey(key)`** — returns `.true.` if the key exists.
- **`dictType%enquireShape(key)`** — returns an `integer :: shp(2)` array containing `[sizem, sizen]` for the stored entry.
- **`dictType%enquireType(key)`** — returns the stored type string. int32 array types are not detected; use int64 arrays for reliable introspection.

Example — querying the shape of a 2D array before allocating a local buffer:

```
integer :: shp(2)
shp = PyObj%enquireShape('coords')
! shp(1) = number of columns, shp(2) = number of rows
```

1.6 Data model

`py2f()` converts each entry in `_kwargs` according to its Python type. The two tables below list all supported types, grouped into Python types and NumPy arrays.

Python types

Python scalars and other built-in types are passed by value. They are immutable: changing the Python attribute after `py2f()` does not update the Fortran side, and Fortran cannot write back to them. To make a value writable from Fortran, use a one-element NumPy array instead (see section 1.4).

Python type	Fortran receives	Notes
`int`	`INTEGER(8)` scalar	Copy by value via `getIntCLong`
`float`	`REAL(8)` scalar	Copy by value via `getDouble`
`bool`	`LOGICAL`	Via `c_bool`, copy by value
`str`	`CHARACTER(len=:), pointer`	Truncated to 256 characters if longer; use `trim()` on the Fortran side to obtain the actual content
`list`	`INTEGER(8)` array	Converted to a NumPy array via `asarray`; if all elements are strings, joined with `;` and passed as a single `CHARACTER`; non-convertible lists are passed as `None`
`tuple`	`INTEGER(8)` array	Same treatment as `list`
`dict`	null `c_void_p`	Not passed to Fortran; treated as `None`
`None`	null `c_void_p`	Placeholder; not accessible from Fortran
callable	`c_funptr`	C function pointer; keep the Python callable alive; see section 1.3
nested object	`PyType`	Recursive `py2f()` conversion; any object with a `_kwargs` attribute qualifies; objects without `_kwargs` are silently skipped; see section 1.4

Any type not listed above is silently skipped by `py2f()`.

NumPy arrays

NumPy array entries are converted to `ndpointer` by `py2f()`. Fortran retrieves them via `dictType%get` as pointers (alias / lazy mode) or copies (safe mode with `readonly=.true.`). Because NumPy arrays are mutable containers, in-place modifications on either side are visible to the other when aliased. Setting `readonly` on getters forces a copy, and mutating that copy will not affect the other side.

NumPy dtype	Fortran receives	Supported ranks	Notes
`numpy.int64`	`INTEGER(8)` array	1, 2	Recommended integer type
`numpy.int32`	`INTEGER(4)` array	1, 2	Deprecated; prefer int64 (see below)
`numpy.float64`	`REAL(8)` array	1, 2	Primary float type; zero-copy when contiguous
`numpy.float32`	`REAL(8)` array	1, 2	Pointer type declared as `c_double` by `py2f()`; ensure the source array is `float64` for correct results
`numpy.S` / char array	`CHARACTER` array	1	Due to 8-byte alignment, Fortran receives only the first character of each element; 2D character arrays are unsupported
structured array (recarray / void)	per-field arrays	1	See below
masked array (`MaskedArray`)	per-field arrays	1	See below

Structured arrays. When `py2f()` encounters a `numpy.recarray` or void-dtype array, it unpacks each named field into a separate Fortran array, each retrievable by name via `dictType%get`. For example, a Z-matrix with fields `j`, `bond`, `k`, `angle`, `l`, `dihedral` becomes six individual 1D arrays on the Fortran side. On the Python side, populate sub-fields with dictionary-style access:

```
my_app.zmatrix['bond']  = 1.5   # or my_app.zmatrix.bond = 1.5
my_app.zmatrix['angle'] = 90.0  # or my_app.zmatrix.angle = 90.0
```

Masked arrays. When the value is a `numpy.ma.MaskedArray`, the data portion is extracted via `.torecords()['_data']` and treated as a structured array. Mask information is passed alongside: for each field, a `LOGICAL` flag indicates whether it belongs to the mask.

Other unrecognised dtypes cause the field to be skipped with a warning.

int32 deprecation note. The `get`/`set` generics retain their int32-specific procedures, but prefer `int64` (`numpy.int64` / `INTEGER(8)`) for new code. The int32 `set` procedures (`setOneDimInt`, `setTwoDimInt`) write into int64 storage internally, which may cause unexpected implicit conversion. int32 paths receive less testing and may be removed in future releases.

Contiguity and strides. `py2f()` passes the raw memory address of the NumPy array (via `.ctypes.data`) without checking strides or contiguity. The developer must ensure arrays

are contiguous before calling `py2f()`; non-contiguous slices or views will result in Fortran reading incorrect data. When using `_mega`, every field in `_fields` must use 8-byte aligned dtypes to keep the `recarray` layout stable across C and Fortran.

```
import numpy

a = numpy.zeros((10, 6), dtype=numpy.float64)

# WRONG: a[:, ::2] is non-contiguous; Fortran sees garbage
app.coords = a[:, ::2]

# RIGHT: make a contiguous copy first
app.coords = numpy.ascontiguousarray(a[:, ::2])
```

Copy semantics

Case	Behaviour
`readonly=.false.` (default)	Fortran receives an alias pointer into the NumPy buffer (zero-copy); changes are reflected on the Python side immediately
`readonly=.true.` on `get`	Fortran receives its own copy; you must call `%set` to push changes back to Python

1.7 Memory ownership

In the Python-to-Fortran direction, Python owns all NumPy arrays and scalar objects; Fortran receives either aliases (zero-copy pointers) or copies depending on the `readonly` flag. DL_PY2F never frees Python-owned memory. The caller must keep the Python object and its NumPy backing arrays alive for the entire Fortran call because DL_PY2F does not extend their lifetime. Fortran-side `dictType` handles point into Python buffers — dropping them (via `finalise` + `deallocate`) releases only the Fortran metadata, not the underlying data. Callback function pointers are stored by Fortran but owned by Python; the caller must keep the Python callable alive.

Ownership matrix

Operation	Data movement	Who owns the buffer aft
`ptr2dict(objPtr)`	None (all entries alias Python)	Python
`dictType%get(key, array, readonly=.false.)`	None (alias)	Python (NumPy)
`dictType%get(key, array, readonly=.true.)`	Copy Python → Fortran	Fortran (caller)
`dictType%get(key, scalar)`	Copy by value	Fortran (local variable)
`dictType%set(key, array)`	Copy Fortran → Python	Python (NumPy)

Note that `finalise` nullifies all pointers inside the `dictType`, but any local Fortran pointer variables that were obtained via `%get` with `readonly=.false.` still point into the NumPy memory:

```
call PyObj%get('coords', twodimdbl)  ! twodimdbl points to NumPy buffer

call PyObj%finalise()                ! nullifies dictType internals
deallocate(PyObj)

! twodimdbl is a local variable, still pointing to the NumPy array
twodimdbl(1,1) = -999.9999999       ! this works but is dangerous
```

This is technically valid but dangerous — the developer should `nullify` local pointers after `finalise` to avoid accidentally writing to Python memory after the intended interaction is complete.

Part 2 — Fortran-to-Python interoperability

Fortran-to-Python interoperability is for applications that already have data and logic in Fortran and want to access them from Python, for example reading module variables, modifying derived-type fields, or driving a Fortran library from a Python script. No changes

to the Fortran source code are needed; DL_PY2F works by loading the compiled `.so` and parsing `.mod` files at runtime. This path currently supports gfortran, ifx, Flang, and nvfortran.

2.1 Getting started

Load the Fortran shared library, point to the directory containing the `.mod` files, and access Fortran variables with Python dot syntax:

```
import dl_py2f

lib = dl_py2f.DL_DL('/path/to/libexample.so')
lib.moddir = '/path/to/modules'

# read a module-level scalar
energy = lib.modules.yourmodule.var01_of_real

# write to a module-level variable
lib.setValue('var01_of_real', 3.14, module='yourmodule')

# access a derived-type instance
dt = lib.getValue('var_of_t04', module='yourmodule')
dt.cvar01 = 'hello'

# access nested arrays of derived types
dt.t2ar02[3,4].t1ar01[1].rvar01 = 3.41
```

All access goes directly into Fortran memory, there are no copies. Writes take effect immediately on the Fortran side. Python objects must not outlive the Fortran allocation; if Fortran later `deallocate`s an array, the Python view becomes invalid.

2.2 Data model

Supported intrinsic types

Fortran type	Python exposure
`INTEGER(4)`	`int` or `numpy.int32`
`INTEGER(8)`	`int` or `numpy.int64`
`REAL(4)`	`numpy.float32` (read/write; may copy)
`REAL(8)`	`numpy.float64`
`COMPLEX(4)`	`c_complex4` struct with `re` and `im` fields (`c_float`)
`COMPLEX(8)`	`c_complex8` struct with `re` and `im` fields (`c_double`)
`LOGICAL`	`bool` (via `c_bool`)
`CHARACTER`	`str` or NumPy `S`
derived type	`DL_DT` proxy with dotted field access

`c_complex4` and `c_complex8` are `ctypes.Structure` subclasses. Access the real and imaginary parts as `val.re` and `val.im`.

Supported array ranks

Type	Supported rank
INTEGER / REAL	Any rank. Shape and strides are read from the compiler's array descriptor
COMPLEX	Any rank
LOGICAL	Scalar only (rank 0)
CHARACTER	Scalar and 1D. Two-dimensional character arrays are unsupported
Derived type	Any depth. Arrays of derived types at any rank are accessible through module parsing

Rank information is extracted from `.mod` metadata and the compiler's array descriptor at runtime, so there is no compile-time rank limit for numeric types.

Supported Fortran attributes

Variables and array members with the `allocatable` or `pointer` attribute are fully supported, including deferred-length characters (`character(len=:), allocatable` or `character(len=:), pointer`). For `pointer` variables, DL_PY2F dereferences the pointer automatically and exposes the target. Pointers must be initialised (e.g., `=> null()`) before DL_PY2F accesses them; uninitialised pointers may cause undefined behaviour.

Linked lists

Fortran derived types that contain a `pointer` member pointing to the same type (i.e., a linked-list node) are fully supported. DL_PY2F dereferences each link lazily — the next node is constructed only when the pointer member is accessed in Python. The entire chain can be traversed by following the pointer member repeatedly:

```
node = dt.head
while node is not None:
    print(node.value)
    node = node.next    # follows the pointer to the next node
```

Traversal stops automatically when a null or unassociated pointer is encountered. This works for arbitrarily deep chains with no limit on the number of nodes.

Arrays, contiguity, and strides

DL_PY2F reads strides and shape from `.mod` metadata and the compiler's array descriptor, then constructs NumPy views that mirror the Fortran memory layout with `copy=False`. No copies are made.

Because Fortran uses column-major (Fortran order) and NumPy defaults to row-major (C order), the dimensions of the array appear reversed on the Python side. For example, a Fortran array declared as `real(kind=8) :: a(3,4)` appears as a NumPy array with shape `(4,3)`. Additionally, Fortran arrays are 1-indexed while NumPy arrays are 0-indexed.

The resulting views (both numeric arrays and arrays of derived types) support standard NumPy indexing, including slicing, negative indices, fancy indexing, `Ellipsis` (`...`), and `None` (newaxis). For example:

```
for elem in dt.t2ar02[:, 2]:
    print(elem.ivar04)
```

For deferred (allocatable or pointer) arrays, DL_PY2F uses the `.mod` metadata to locate the array descriptor within the shared library's data segment, then reads the descriptor at runtime to determine the current shape, strides, and bounds. Matching NumPy views are constructed automatically when you access the array with Python dot syntax.

Memory ownership

In the Fortran-to-Python direction, the ownership model is the reverse of the Python-to-Fortran direction (section 1.7): Fortran owns all variables and arrays, and Python receives thin views that point directly into the Fortran data segment. These views take two forms:

- **Numeric arrays** are exposed as NumPy arrays constructed with `copy=False`.
- **Derived types** are exposed as `DL_DT` proxy objects (see section 2.4).

Dropping a Python view or `DL_DT` object does not free Fortran memory. However, if Fortran deallocates the underlying storage or the shared library is unloaded, the Python view becomes invalid. In practice the `.so` stays loaded for the lifetime of the process, so views remain valid as long as Fortran does not deallocate. DL_PY2F never allocates or frees Fortran memory.

Unsupported cases. Two-dimensional `CHARACTER` arrays, non-contiguous Fortran layouts that cannot be described from `.mod` metadata, and compiler-specific kinds beyond those listed above.

2.3 Initialisation and teardown

The loading sequence is shown in section 2.1. Additional notes:

- Loading a library and parsing modules can be called multiple times on the same instance without side effects; repeated calls reuse the cached metadata. To force re-parsing, use `parseAllModules(..., overwrite=True)`.
- Multiple `DL_DL` instances for different libraries can coexist in the same process.
- There is no mandatory teardown beyond normal Python garbage collection. The `.so` stays loaded for the lifetime of the process. Fortran finalisation remains Fortran's responsibility.

2.4 API reference

DL_DL — shared-library loader

- **`DL_DL(fullpath_to_lib, mode=RTLD_GLOBAL, debug=False)`** Loads a Fortran `.so` and creates a binding context. `DL_DL` inherits from `ctypes.CDLL`, so all standard CDLL methods are available. The `mode` argument controls symbol visibility (default `RTLD_GLOBAL` makes

symbols available to subsequently loaded libraries). It fails if the library is missing or if symbols cannot be resolved.

- **`dl.symbols`** (property) Returns a list of exported symbol names. Internal Fortran helper symbols (copy, deallocate, vtab, etc.) are filtered out automatically.
- **`dl.modulenames`** (property) Extracts unique Fortran module names from the symbol table.
- **`dl.modules`** (property) Returns a dictionary-like object supporting attribute-style access to parsed Fortran modules (e.g., `dl.modules.my_mod.my_var`). Parsing is lazy: the first access triggers the parse of all `.mod` files registered via `moddir`.
- **`dl.moddir`** (property / setter) Registers one or more directories containing `.mod` files. No ownership is transferred. If the path does not exist, a warning is printed but no exception is raised. Fortran submodule files (`.smod`) are not tested and may not be supported.
- **`dl.searchSymbol(*keywords)`** Case-insensitive multi-keyword search over the symbol table. Returns all symbols whose names contain every keyword.
- **`dl.parseModule(modpath, padding=True, debug=False)`** Parses a single `.mod` file (gfortran gzip format, Flang text format, or nvfortran V35 ASCII format; auto-detected). Returns a dictionary with per-variable entries describing their type, shape, and layout. The `padding` argument controls whether struct-alignment padding is applied; the default `True` is correct for most cases and should not normally be changed.
- **`dl.parseAllModules(moddir, recursive=True, padding=True, overwrite=False)`** Walks a directory tree for `.mod` files and calls `parseModule` on each. Results are merged into `dl.modules`. Setting `overwrite=True` replaces previously parsed entries.
- **`dl.getValue(symbol, ctype=c_int, shape=(), module='', return_ctype=False, debug=False)`** Retrieves a variable, array, or derived-type handle. Returns a view rather than owning the memory. Supports `%`-separated member access paths for derived-type fields (e.g., `'my_type%coords'`). It fails if the symbol is not found or the type or layout is unsupported.
- **`dl.setValue(symbol, value, module='', debug=False)`** Writes a value into a Fortran variable, mutating Fortran state directly. It fails on type or shape mismatches or when the target is read-only.

Example: `python v = lib.getValue('var01_of_real', float, module='yourmodule')`
`lib.setValue('var01_of_real', 3.14, module='yourmodule')`

Byte-level setters (static methods on `DL_DL`)

These static methods write a value directly into Fortran memory at a given address:

Method	Target type	Width
`DL_DL.setInt(entity, val)`	`c_int`	4 B
`DL_DL.setLong(entity, val)`	`c_long`	8 B
`DL_DL.setChar(addr, length, val)`	`c_char`	variable
`DL_DL.setBool(entity, val)`	`c_bool`	1 B
`DL_DL.setFloat(entity, val)`	`c_float`	4 B
`DL_DL.setDouble(entity, val)`	`c_double`	8 B

DL_DT — derived-type proxy

`DL_DT` is a Python proxy for a Fortran derived-type instance. It does not own the Fortran memory; it provides read and write access to the fields of the type via Python's dot syntax. Given the following Fortran module:

```
module yourModule

    type type_01
        real(kind=8) :: rvar01 = 1.0
    endtype type_01

    type type_02
        integer         :: ivar04 = 1004
        type(type_01)   :: t1ar01(3)
        character(len=25) :: cvar01 = 'char 01'
    endtype type_02

    type type_04
        character(len=25)          :: cvar01
        integer(kind=8)            :: iarr02(2,3,4)
        type(type_02), allocatable :: t2ar02(:,:)
    endtype type_04

    type(type_04) :: var_of_t04

endmodule yourModule
```

you can access its fields from Python as follows:

```
dt = lib.getValue('var_of_t04', module='yourmodule')

# read and write scalar fields
dt.cvar01 = 'dl_py2f'

# access multi-dimensional arrays
dt.iarr02[1,2,3] = 123

# navigate nested derived types
dt.t2ar02[3,4].t1ar01[1].rvar01 = 3.41

# iterate over arrays of derived types
for elem in dt.t2ar02[:, 2]:
    print(elem.ivar04)
```

Key behaviours:

- **Lazy and cached.** Nested derived-type members are constructed on first access. Accessing the same member repeatedly returns the same proxy object without constructing a new one.
- **Arrays of derived types** support multi-dimensional indexing, iteration, `len()`, and properties `shape`, `ndim`, `dtype`, `size`, `T`, and `__array__`. Python-style slicing (negative indices, `Ellipsis`, `None`) is also supported.
- **Pointer members** are dereferenced automatically. Linked-list structures (where a pointer member points to the same type) can be traversed by accessing the pointer member repeatedly (e.g., `node.next.next`); traversal stops when a null or unassociated pointer is encountered.
- **`dt.setValue(name, value)`** — sets a member value by name, dispatching to the appropriate byte-level setter based on the member's type.

Part 3 — Shared reference

3.1 Error handling and diagnostics

Python-to-Fortran errors

- Fortran side: Requesting a key that does not exist terminates the process for scalar getters and callback getters (Fortran has no exception mechanism). Nested-object getters (`getPyPtr`) and query routines (`enquireShape`, `enquireType`) print a warning but do not terminate. This inconsistent behaviour will be improved in a future version.
- Python side: In `py2f()`, an unsupported type in `_kwargs` prints a message to stdout and the field is skipped.

Fortran-to-Python errors

- Python side: `DL_DL` raises `RuntimeError` when a symbol is not found.

Diagnostic tools

- Use `dictType%keys()` to list all stored keys (Python-to-Fortran path).
- Use `enquireShape` to confirm array dimensions before calling `get` (Python-to-Fortran path).
- Use `enquireType` to inspect the stored type string; note that int32 array types are not detected (Python-to-Fortran path).
- Set `debug > 2` in `py2f()` to trace the conversion process (Python-to-Fortran path).

3.2 Compatibility and limitations

Compiler support

Path	gfortran	ifort / ifx	flang	nvfortran
Python-to-Fortran	Yes	Yes	Yes	Yes
Fortran-to-Python	Yes	Yes	Yes	Yes

The Fortran-to-Python path supports `.mod` files generated by all four compiler families.

Toolchain and dependency requirements

Tools & Libraries	Min. version	Note
g++/gfortran	11	
OR icpc/ifort	17	
OR icpx/ifx	2024	
OR clang++/flang	22	
OR nvc++/nvfortran	26.1	
cmake	3.16	
python3-dev	3.8	
python3-numpy	1.21.5	

The package names above are Ubuntu-based; they may vary on other operating systems.

Installation See Getting DL_PY2F for instructions on installing from source, Debian PPA, or PyPI.

Runtime requirements

- NumPy ≥ 1.21.5. Older versions may lack the `__array_interface__` fields needed for zero-copy. Zero-copy is a best-effort feature.
- MPI: DL_PY2F itself never calls `MPI_Init` or `MPI_Finalize`; see section 3.3.

Known limitations

- **Character arrays.** Due to 8-byte alignment, only the first character of each element in a `numpy.S` array reaches Fortran. 2D character arrays are not supported.
- **int32 partially tested.** The int32 paths remain functional but receive less testing than int64. Prefer int64 for new code.
- **Not thread-safe.** The internal data structures (e.g., the `dictType` linked list on the Fortran side, the struct cache on the Python side) have no locking. If multiple threads access the same `dictType` handle or call `py2f()` concurrently, data corruption or crashes can occur. All access to shared handles must be serialised by the application developer. This is a common limitation of Fortran data structures in general, as the language has no built-in concurrency primitives such as mutexes.
- **Callback exceptions.** If a Python exception is raised inside a callback, it must be caught within the callback itself. An uncaught exception cannot propagate back through Fortran and will crash the program.
- **Complex types.** Supported on the Fortran-to-Python path via struct wrappers (`c_complex4`, `c_complex8`). Complex arrays are not yet supported on the Python-to-Fortran path but will be added in a future version.
- **Unknown dtypes.** Record and void dtypes (structured arrays) are mapped to `REAL(8)` views; other unrecognised dtypes cause the field to be skipped with a warning.

- **Strided views.** `py2f()` passes the raw memory address without checking strides. Non-contiguous NumPy views will produce incorrect data on the Fortran side; ensure arrays are contiguous before calling `py2f()`.
- **Scalar getter leak.** The scalar getters (`getInt`, `getIntCLong`, `getReal`, `getDouble`, `getLogical`, `getCLogical`, `getChar`) leak a small amount of memory per call due to a workaround for a gfortran limitation. The leak is almost negligible: each call leaks roughly 32–64 bytes, so even an MD simulation that invokes scalar getters one million times would accumulate only about 30–60 MB, which is insignificant compared to the memory footprint of a typical simulation.

3.3 MPI / multi-instance usage

DL_PY2F itself does not call `MPI_Init` or `MPI_Finalize`, nor does it manage MPI communicators. MPI lifecycle and communicator management are entirely the application developer's responsibility. Two patterns are supported, and the choice between them depends on the trade-off between simplicity and memory efficiency.

Model A — one Python interpreter per MPI rank

In this model every MPI rank launches its own Python interpreter and loads the same Fortran shared library independently. Each rank has its own copy of the Python object, its own `py2f()` handle, and its own `dictType` on the Fortran side. Each rank's DL_PY2F state and buffers are independent; cross-rank data exchange is handled by the application developer via MPI as usual.

This is the simplest approach: there are no cross-rank pointer hazards, no shared-state coordination, and the standard DL_PY2F workflow (sections 1.1–1.2) applies on each rank without modification. The cost is that Python start-up and memory usage scale with the number of ranks. Launch with:

```
$ mpirun -np 2 python3 user_script.py
```

A minimal Fortran interface for this model:

```
integer(c_int) function interface_app(objPtr) bind(c)

    use iso_c_binding
    use mpi
    use DL_PY2F, only: PyType, dictType, ptr2dict

    type(PyType)           , intent(in) :: objPtr
    type(dictType), pointer             :: PyObj
    real(kind=8)  , pointer             :: forces(:,:)
    integer                             :: ierror

    ! initialise MPI
    call MPI_INIT(ierror)

    ! convert handle and alias forces array
    allocate(PyObj, source=ptr2dict(objPtr))
    call PyObj%get('forces', forces)

    call do_work(forces)

    ! teardown
    call PyObj%finalise()
    deallocate(PyObj)
    nullify(forces)

    call MPI_FINALIZE(ierror)

endfunction interface_app
```

Each rank has its own independent `py2f()` handle and `dictType`. In this example `MPI_INIT` and `MPI_FINALIZE` are placed inside the interface function for simplicity, but they can equally be handled elsewhere (e.g., via `mpi4py` on the Python side, or in the existing Fortran codebase). Cross-rank communication is handled via standard MPI calls as usual.

Model B — single Python interpreter with MPI distribution

In this model only rank 0 runs the Python interpreter. A compiled binary (`app.x`) embeds the Python interpreter on rank 0 and runs a pure MPI replica loop on all other ranks. The key mechanism is a C function-pointer broadcast: rank 0 registers a Fortran wrapper function via `c_funloc`, stores the pointer in a C-level global, and broadcasts it to all ranks so that every rank enters the same Fortran wrapper simultaneously.

This saves memory because Python runs on only one rank. This model also extends to support taskfarming parallelism, where each workgroup has its own Python interpreter and MPI ranks, as used in ChemShell. The trade-off is that the developer must coordinate the broadcast explicitly: never call the wrapper directly on rank 0 without broadcasting first, and if multiple wrappers are needed, a more elaborate dispatch mechanism is required (e.g., a registry mapping task IDs to function pointers, as used in ChemShell). Launch with:

```
$ mpirun -np 2 app.x user_script.py
```

A typical implementation involves three source files in addition to the application's Fortran code. In this example the MPI operations are placed in C for simplicity, but they can equally be implemented in Fortran:

```
my_project/
├── main.c         # C main program (embeds Python on rank 0)
├── parallel.c     # C helper for function-pointer broadcast
├── app.f90        # Fortran module: registration, interface, wrapper
├── my_code/       # my existing Fortran application code
├── dl_py2f/       # DL_PY2F library
├── CMakeLists.txt # builds app.x binary
└── user_script.py # Python entry point (runs on rank 0 only)
```

The C main program initialises Python on rank 0, runs the user script, and coordinates the broadcast:

main.c

```c
int main(int argc, char **argv) {

    // initialise MPI first (all ranks)
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // register the Fortran wrapper (all ranks)
    extern void register_app(void);
    register_app();

    if (rank == 0) {
        // root: start Python, run the user script
        Py_Initialize();
        if (argc > 1) PyRun_SimpleFile(fopen(argv[1], "r"), argv[1]);

        // signal replicas to stop
        parallel_c_end_replicas();

        Py_Finalize();
    } else {
        // replicas: wait for broadcasts from root
        parallel_c_run_replicas();
    }

    MPI_Finalize();
}
```

The Fortran module registers the wrapper at startup, provides the `bind(c)` entry point that Python calls via ctypes, and defines the wrapper that all ranks execute after the broadcast:

app.f90

```
module AppModule

    use iso_c_binding
    implicit none

    ! C functions defined in parallel.c
    interface
        integer(c_int) function parallel_c_exec_root() &
            bind(c, name='parallel_c_exec_root')
        endfunction parallel_c_exec_root

        integer(c_int) function parallel_c_get_comm() &
            bind(c, name='parallel_c_get_comm')
        endfunction parallel_c_get_comm

        subroutine parallel_c_register(func) &
            bind(c, name='parallel_c_register')
            use iso_c_binding
            type(c_funptr), value :: func
        endsubroutine parallel_c_register
    endinterface

    contains

    ! called from Python via ctypes; triggers broadcast + wrapper
    subroutine interface_app() bind(c)
        integer(c_int) :: ierror
        ierror = parallel_c_exec_root()
    endsubroutine interface_app

    ! executed on ALL ranks after broadcast
    subroutine wrapper_app()
        use mpi
        integer(c_int) :: comm_world
        comm_world = parallel_c_get_comm()
        call actual_work(comm_world)
    endsubroutine wrapper_app

    ! called once at startup; stores wrapper_app pointer in C
    subroutine register_app() bind(c, name="register_app")
        use iso_c_binding
        type(c_funptr) :: wrap_pointer
        wrap_pointer = c_funloc(wrapper_app)
        call parallel_c_register(wrap_pointer)
    endsubroutine register_app

endmodule AppModule
```

The C helper manages the function-pointer broadcast between root and replicas:

parallel.c

```c
#include <mpi.h>

// file-scope global holding the wrapper function pointer
static void (*func_ptr)(void) = NULL;

// called from Fortran register_app to store the pointer
void parallel_c_register(void (*func)(void)) { func_ptr = func; }

// broadcast func_ptr from root to all replicas
static int parallel_c_broadcast(void) {
    MPI_Bcast(&func_ptr, sizeof(func_ptr), MPI_BYTE, 0, MPI_COMM_WORLD);
    return func_ptr ? 0 : -1;
}

// root: broadcast pointer then call wrapper (called from Python via ctypes)
int parallel_c_exec_root(void) {
    parallel_c_broadcast();
    if (func_ptr) func_ptr();
    return 0;
}

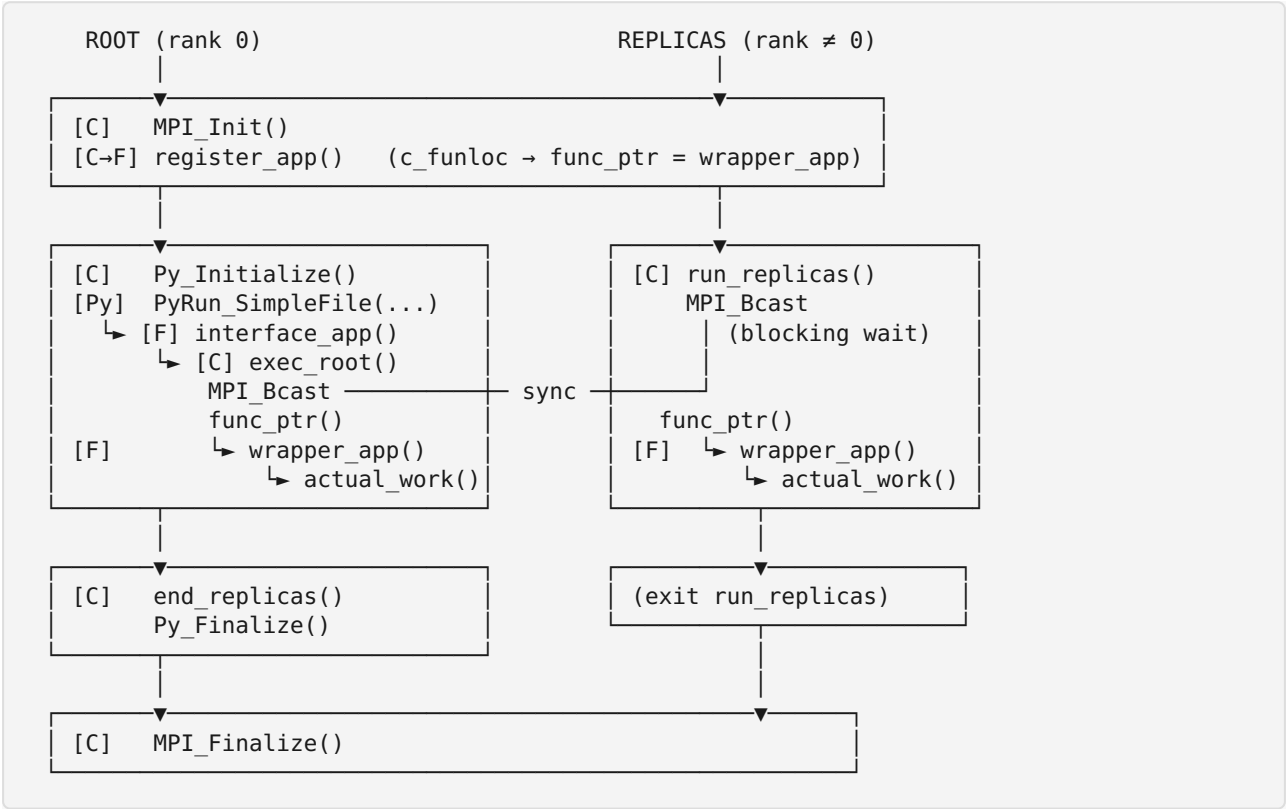
// root: broadcast NULL to signal replicas to exit
void parallel_c_end_replicas(void) {
    func_ptr = NULL;
    parallel_c_broadcast();
}

// replicas: loop waiting for broadcasts; exit when NULL is received
void parallel_c_run_replicas(void) {
    while (1) {
        parallel_c_broadcast();
        if (!func_ptr) break;
        func_ptr();
    }
}

MPI_Comm parallel_c_get_comm(void) { return MPI_COMM_WORLD; }
```

The following diagram illustrates the complete execution sequence for Model B. All ranks initialise MPI and register the wrapper together. Then root starts Python and runs the user script, while replicas enter a loop waiting for broadcasts. Each time Python calls `interface_app()`, root broadcasts the wrapper pointer and all ranks execute it. The replicas remain in the loop, ready for the next broadcast, so `interface_app()` can be called any number of times. Root signals replicas to exit the loop by broadcasting a null pointer (`end_replicas`).

Model B sequence: Legend: `[C]` = C code, `[F]` = Fortran code, `[Py]` = Python interpreter



General notes

- **Multiple wrappers.** For more complex scenarios involving multiple applications or repeated invocations, a more elaborate coordination mechanism is needed. For example, ChemShell uses a linked-list registry mapping application IDs to function pointers, broadcasts the task ID instead of the pointer itself, keeps the replica loop running indefinitely, and provides an explicit exit signal to terminate it.
- **MPI lifecycle.** DL_PY2F does not virtualise MPI communicators, nor does it call `MPI_Init` or `MPI_Finalize`. The timing of handle creation and destruction is entirely the application's responsibility.
- **Thread safety.** DL_PY2F contains no locks. When using MPI with multithreading, all access to shared handles must be serialised by the application developer.
- **Quick launch.** Model A: `mpirun -np 2 python3 user_script.py`. Model B: `mpirun -np 2 app.x user_script.py`. Note that the `main.c` example above passes `argv[1]` directly to `PyRun_SimpleFile`; in production, the developer should add proper command-line argument handling.

3.4 Common pitfalls and reminders

Python-to-Fortran pitfalls

- Aliased arrays fetched with `readonly=.false.` must never be `deallocate`d on the Fortran side; use `nullify` only. See section 1.7 for the full ownership matrix.
- When using `py2f`, ensure `_kwargs` entries use supported types (see section 1.6). Unsupported types are silently skipped.

- Scalars (`int` , `float` , `bool`) in `_kwargs` are immutable and cannot be written back from Fortran. Use a one-element array instead for any value that Fortran needs to modify (see section 1.4).
- With nvfortran, integer and real arrays retrieved via `%get` with the default `readonly=.false.` may produce corrupt data due to a compiler bug with non-contiguous pointer arrays. The workaround is to use `readonly=.true.` (safe mode) and then `%set` to write changes back. Use the C preprocessor `#ifdef __NVCOMPILER` guard to apply this selectively:

```
#ifdef __NVCOMPILER
! nvfortran workaround: force a copy with readonly=.true.
call PyObj%get('tags', tags, readonly=.true.)
tags = tags + 100
call PyObj%set('tags', tags)
#else
! gfortran/ifort/ifx/flang: direct read/write access
call PyObj%get('tags', tags)
tags = tags + 100
#endif
```

Fortran-to-Python pitfalls

- On the Fortran-to-Python path, `DL_DT` proxies are cached; dropping all Python references invalidates the cache entry, not the Fortran memory.

General

- Always keep the owning Python object alive for as long as any Fortran pointer, callback, or aliased array may be used. DL_PY2F does not extend lifetimes.
- If the existing Fortran codebase has memory leaks, they will accumulate on every call to the interface function. This is not a DL_PY2F issue, but it can become critical in applications that call the interface many times (e.g., molecular dynamics), especially under MPI where each rank leaks independently.
- A single process can use DL_PY2F with multiple Fortran shared libraries simultaneously (e.g., multiple `DL_DL` instances or multiple `py2f()` handles for different objects), provided the Fortran libraries do not have conflicting symbols.

3.5 Utility modules (`dl_py2f.utils`)

DL_PY2F ships a set of utility modules under `dl_py2f.utils` . The modules listed below are those used by the library itself or expected to be useful to application developers. Import them as, for example, `from dl_py2f.utils import objutils` .

`objutils` — object initialisation

- **`@init(inherit=(), init=(), priorities=(), inheritfrom=None)`** — decorator for `__init__` methods. Automates attribute inheritance from base classes and initialisation from `_kwargs` / `_internals` dictionaries. See section 1.4 for advanced class features.
- **`inheritAttrs(obj, inherit, inheritfrom)`** — merges sequence and dict attributes from base classes into `obj` .
- **`initAttrs(obj, init, priorities)`** — processes each attribute dictionary listed in `init` , respecting `priorities` ordering. Delegates to property setters where defined.

- **`checkType(obj, cls, info, chksubcls)`** — checks or converts `obj` to an instance of `cls`. Raises `TypeError` on failure.
- **`getDeepCopy(obj)`** — returns a deep copy of `obj`, handling NumPy arrays and nested structures.
- **`getType(obj)`** — returns the type of `obj` as a string, distinguishing NumPy arrays, ctypes objects, and plain Python types.
- **`obj2dict(obj)`** — converts an object's attributes to a dictionary.
- **`setAttr(obj, attr, val)`** — sets a possibly nested attribute (e.g., `'child.coords'`) on `obj`.
- **`getAttr(obj, attrname)`** — retrieves a possibly nested attribute from `obj`.

`nputils` — NumPy helpers

- **`RecArray`** — a `numpy.recarray` subclass used as the backing store for `_mega`. Key methods:
- `expand(N, axis=0)` — resizes the array to `N` elements, copying the first element into new slots.
- `setField(buff, field=None, factor=1.0)` — writes values into a named field in place, with optional scaling.
- `getDtype(field)` — returns the dtype of a named field.
- **`SymArray`** — a `numpy.ndarray` subclass for symmetric matrices; assignments to `(i,j)` are automatically mirrored to `(j,i)`.
- **`getDTypeFromObj(obj, dictname='_kwargs')`** — constructs a `numpy.dtype` from the fields of a data-class object. Used when building `_mega`.
- **`setField(a, buffer)`** — standalone function; writes `buffer` into array `a` in place.
- **`bytes2str(arr)`** — converts a NumPy byte-string array to a list of Python strings.
- **`tolist(arr)`** — like `ndarray.tolist()` but converts byte strings to `str`.

`modutils` — shared-library loading

- **`getSharedLib(modname, location, mode=RTLD_GLOBAL, debug=0)`** — loads a shared library (`.so`) via `ctypes.CDLL`. Returns the library handle or `None` on failure.
- **`getLinkedLib(modname, location, mode=RTLD_GLOBAL, is_linked=False)`** — loads a shared library only if it is already linked into the process.
- **`isLinked(modname, location)`** — returns `True` if the named library is linked. This works by loading the `.so` and calling an `isLinked()` function that must be compiled into it. The application developer must include a minimal Fortran file (e.g., `linked.f90`) in their shared library: `fortran integer(c_int) function isLinked() result(linked) bind(c) use iso_c_binding linked = 1 endfunction isLinked`
- **`importModule(modname, filepath, relative_dir='.', suppress_warning=True)`** — dynamically imports a Python module from a file path.

`fileutils` — file operations

- **`archiveFile(filename, dir='.', copy=False, suffix='.OLD')`** — moves (or copies if `copy=True`) a file to `filename.OLD` (or the given suffix). Overwrites the destination silently if it already exists.
- **`archiveDir(dirname, suffix='.OLD', mkdir=True)`** — moves a directory to `dirname.OLD`. Overwrites the destination silently if it already exists. If `mkdir=True`, recreates the original directory after moving.
- **`file2text(filename, case='input')`** — reads a file and returns its contents as a string, optionally converting case.
- **`getChoppedMasks(filename, section_key)`** — extracts named sections from a text file delimited by `section_key` markers.

`miscutils` — miscellaneous helpers

- **`getCaller(levels=1)`** — returns the name and location of the calling function, useful for diagnostic messages.
- **`addWeakRef(obj)`** — attaches a weak-reference callback to `obj` for finalisation tracking.
- **`runSysCmd(syscmd, stdinp='', stdout='', stderr='', shell=True)`** — executes a shell command via `subprocess` and returns its output.

`strutils` — string helpers

- **`str2num(s)`** — parses a numeric string into `int`, `float`, or a NumPy array.
- **`bytes2str(b)`** — decodes a `bytes` object to `str`.
- **`str2bytes(s)`** — encodes a `str` to `bytes`.
- **`inOrEq(s, ref)`** — case-insensitive membership or equality test.

`dictutils` — dictionary helpers

- **`getDictEntry(dictObj, *args)`** — retrieves a value from a nested dictionary by following the chain of keys.
- **`hasDictEntry(dictObj, key1, key2=None)`** — tests whether a nested dictionary contains the given key path.
- **`getSortedKeys(dictObj, order=None)`** — returns dictionary keys sorted by a priority ordering, then alphabetically.
- **`mergeDicts(d1, d2)`** — recursively merges d2 into d1; array values are appended rather than replaced.
- **`getValue(d, key, case_sensitive=True, raiseError=True)`** — retrieves a value with optional case-insensitive key matching.
- **`isKeyOf(d, key, case_sensitive=True)`** — tests whether a key exists in the dictionary, with optional case insensitivity.

`iterutils` — iteration and flattening helpers

- **`getFlattened(iter, unique=True)`** — flattens nested lists and tuples into a single list, optionally removing duplicates.

- **`getFlattenedIter(iter, sort=True)`** — flattens nested iterables using `itertools.chain`, with optional deduplication.
- **`findMapped(iter, mapping, case_sensitive=True)`** — finds elements of an iterable that appear as keys in a mapping.

`ctypeutils` — ctypes byte conversion

Low-level converters used internally by the Fortran-to-Python path (`DL_DL.setValue`). Application code rarely needs these directly.

- **`int2intarray(i)`**, **`long2intarray(l)`**, **`float2intarray(f)`**, **`double2intarray(d)`**, **`bool2intarray(b)`** — convert a Python scalar to a ctypes-compatible byte array for writing into Fortran memory at a given address.

`colours` — ANSI terminal colour constants

Provides named ANSI escape-sequence constants (e.g., `_R` for red, `_G` for green, `CLR_` for reset) used internally by DL_PY2F for coloured debug output. Application code rarely needs these directly.